Лабораторная работа №7
Применение ORM для доступа к данным БД

1Цель работы

1.1 Научиться создавать приложение C# для организации доступа к БД.

1.2 Научиться проектировать приложение, использующее паттерн репозиторий и Dapper.

2Литература

2.1 https://metanit.com/sharp/aspnet5/26.1.php

2.2 https://learn.microsoft.com/ru-ru/dotnet/framework/data/adonet/asynchronous-programming

3Задание

При выполнении заданий:

- структурировать файлы проекта по папкам,

- использовать рекомендации IDE по улучшению кода (например, заменять конструктор на основной конструктор),

- при вызове методов в основном приложении перехватывать исключения и сообщать о них пользователю.

3.1 Создание классов для работы с БД

3.1.1 Добавить классы модели данных для представления сущностей Посетитель и Жанр с открытыми автосвойствами:

- если значение свойства допускает null, указывать его тип как Тип?
- если значение свойства не допускает null, присваивать ему null!;

3.1.2 Создать открытый класс DatabaseContext, управляющий подключением к БД:

```
public class DatabaseContext
{
    private readonly string _connectionString;

    public DatabaseContext(string server, string database, string login, string password)
    {
        // составление строки подключения _connectionString
    }

    public IDbConnection CreateConnection() => new SqlConnection(_connectionString);
}
```

Провести рефакторинг, создав основной конструктор вместо конструктора с параметрами.

3.1.3 Создать в консольном приложении объект типа DatabaseContext. Проверить, что подключение открывается, используя метод Open() после создания подключения.

3.2 Создание и реализация интерфейса репозитория

3.2.1 Создать обобщенный интерфейс репозитория, определяющий основные операции для работы с сущностями в БД:

```
public interface IRepository<T> where T : class
{
    Task<T> GetByIdAsync(int id);
    Task<IEnumerable<T>> GetAllAsync();
    Task<int> AddAsync(T entity);
    Task UpdateAsync(T entity);
    Task DeleteAsync(int id);
}
```

Заменить тип Task<T> на Task<T?>, допускающий null при возврате значения по идентификатору.

3.2.2 Создать классы репозитория *Посетитель*Repository и *Жанр*Repository (будут обеспечивать чтение и запись данных таблиц Посетитель и Жанр).

Добавить в них поле _dbContext и конструктор, присваивающий значение полю:

```
private readonly DatabaseContext _dbContext;

public UserRepository(DatabaseContext dbContext)
{
    _dbContext = dbContext;
}
```

Заменить поле и конструктор на первичный конструктор, используя рефакторинг.

3.2.3 Указать, что классы репозитория реализуют интерфейс IRepository<T>. Используя рефакторинг, сгенерировать реализацию методов интерфейса для работы с классами модели данных.

3.2.4 Создать в консольном приложении объекты типа классов репозитория.

3.3 Реализация методов на чтение данных

3.3.1 Реализовать метод GetByIdAsync в классах репозиториев. Метод должен возвращать объект по идентификатору.

Для реализации использовать метод Dapper QuerySingleOrDefaultAsync<Тип>.

3.3.2 Реализовать метод GetAllAsync в классах репозиториев. Метод должен возвращать список всех объектов.

Для реализации использовать метод Dapper QueryAsync<Тип>.

3.3.3 Проверить работу созданных методов, вызвав их в консольном приложении.

3.4 Реализация методов на добавление данных

3.4.1 Реализовать метод AddAsync в классах репозиториев. Метод должен возвращать идентификатор добавленного объекта.

Для реализации использовать метод Dapper ExecuteScalarAsync<int>.

Для получения сгенерированного идентификатора записи можно использовать OUTPUT INSERTED.СтолбецИдентификатора, как представлено в следующей команде:

```
INSERT INTO Таблица(столбцы)
OUTPUT INSERTED.СтолбецИдентификатора
VALUES(значения столбцов);
```

3.4.2 Проверить работу созданных методов, вызвав их в консольном приложении.

3.5 Реализация методов на редактирование и удаление данных

3.5.1 Реализовать метод DeleteAsync в классах репозиториев. Метод должен удалять объект из БД по идентификатору.

Для реализации использовать метод Dapper ExecuteAsync.

3.5.2 Реализовать метод UpdateAsync в классах репозиториев. Метод должен изменять данные объекта с идентификатором объекта.

Для реализации использовать метод Dapper ExecuteAsync.

3.5.3 Проверить работу созданных методов, вызвав их в консольном приложении.

4Порядок выполнения работы

4.1 Выполнить все задания из п.3 в консольном приложении на C#.

4.2 Ответить на контрольные вопросы.

5Содержание отчета

5.1 Титульный лист

5.2 Цель работы

5.3 Ответы на контрольные вопросы

5.4 Вывод

6Контрольные вопросы

6.1 Что такое Dapper и для чего используется?

6.2 Какие методы Dapper позволяют извлечь данные из БД?

6.3 Какие методы Dapper позволяют изменить данные в БД?

6.4 Зачем используетсяя паттерн «репозиторий»?